

# Fail-closed class dispatch

**Class-admitted work, data-plane bind. This machine has no leftover-hop edge.**

Rogue Briceño · Version 0.8.1 · 4 September 2026 · CC BY 4.0 · Reference implementation 0.5.0.

## Abstract

A specialized model fleet is a checkable claim only if a work item of class  $c$  cannot complete a stage on a specialist or model outside the policy for  $c$ . Fail-closed class dispatch is that check: a work item carries a class and a frozen body; a policy gives each class an allow set  $\pi(c)$  and a deny set  $\delta(c)$ ; and each specialist  $a$  is bound to one API model identity  $\phi(a)$ . A stage may run only after a well-formed admit of some  $a \in \pi(c) \setminus \delta(c)$ , and only with the executed model recorded by `Observe`; if that bind cannot be served, the stage fails closed and the failure is published. Retry, where there is one, is the same item, the same class, a specialist not yet tried.

This is Clark–Wilson integrity applied to LLM binds, not a new access-control algebra: allow/deny, fail-safe defaults and dual control are old. The added obligations are three —  $\phi$  is the identity `Observe` recorded from the inference client, leftover fallback is not an edge in this table, and the event contract is the witness set. An accidental hop is in scope; an adversarial worker that forges `Observe` is not.

Safety invariants I1–I6, I8 and I9 hold on the abstract stage machine under A0–A9; I7 bounds the Admit count; and under A10–A13, I10–I17 prove FCD-owned work/envelope pinning, package and receipt binding, steering scope, attempt-local cache identity and accepted-only memory promotion. Item liveness and model quality are not proved.

## 1. Problem

Routing, cascades and mixture-of-agents optimize score or spend (Chen et al., 2023; Ong et al., 2024; Wang et al., 2024); mixture-of-experts is a learned gate inside one network (Jacobs et al., 1991; Shazeer et al., 2017); orchestration says who calls whom. None of them forbids a hop to an unbound model when the bound provider is down, and none of them treats a model that refuses a class as illegal rather than low-scoring.

The sentence “we run specialists” is then unfalsifiable: the control plane can name one model while the data plane calls another; a 403 can look like work still in progress; a reviewer can be the author. The required property is class integrity — a pass record lies in  $\pi(c) \setminus \delta(c)$  and uses  $\phi(a)$ .

## 2. Objects

A **work item** has class  $c$ , charge, frozen body hash, author set, required-stage list `Required(c)`, and status `open` | `failed` | `accepted`.  $\pi(c)$  and  $\delta(c)$  are disjoint; on a check stage the effective allow set is  $\pi\_chk(c, authors) = \pi(c) \setminus authors$ .  $\phi: A \rightarrow M$  maps a specialist to an API identity, and display strings are compared after norm.  $u(m)=0$  iff bind time returns 401, 403, 404, 429, exhausted, or `not_found`, and `tried` is the set of specialists already admitted on this stage.

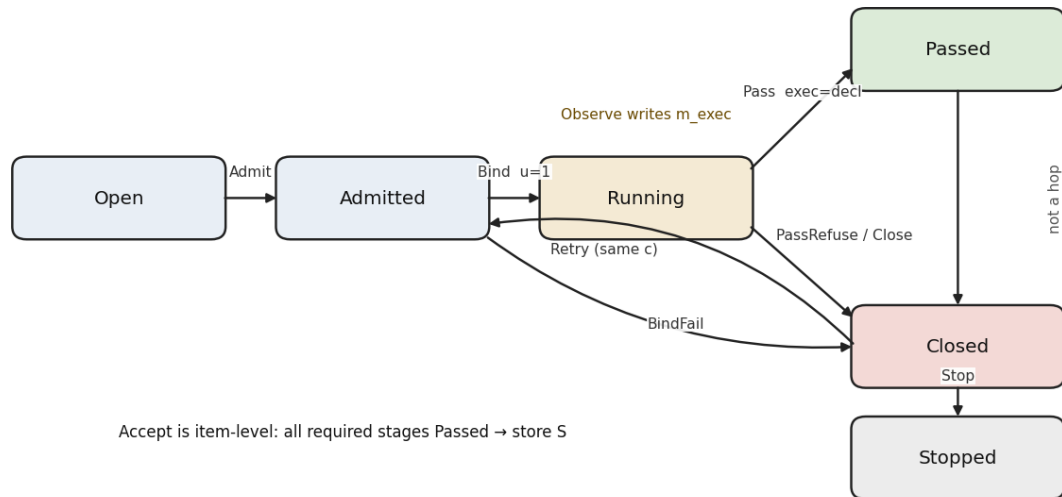
A stage is well-formed only if a control event names class, specialist, declared model and body hash; prose that mentions a name is not a stage. Fail closed means publishing a `fail_closed` decision, after which the next move is `ask`, `retry` in  $\pi(c) \setminus \delta(c) \setminus tried$ , or `stop`. An **accepted artifact** exists only when every stage in `Required(c)` has passed, and the store accepts only accepted artifacts.

### 3. Process

1. Open a well-formed work item.
2. Admit  $a \in \pi^*(c) \setminus \delta(c) \setminus \text{tried}$ . If none, fail closed.
3. Bind. Writes  $m\_decl = \phi(a)$ . If  $u(\phi(a))=0$ , fail closed. Does not write  $m\_exec$ .
4. Observe. Copies  $m\_exec$  from the provider call (A1).
5. Pass only if  $\text{norm}(m\_exec)=\text{norm}(m\_decl)$ ; else PassRefuse (F1).
6. When  $\text{Required}(c)$  is covered, Accept into the store.

A death watchdog outside the worker records death while Running (A2, A9). Close publishes.

Figure 1. Stage automaton (Observe machine)



### 4. What holds

Bind writes the declared identity only and Observe writes the executed one, so I1 is non-vacuous; under A0–A9, I1–I6, I8 and I9 are inductive, and I7 bounds the Admit count. Ask may idle. The leftover-hop picture is an illustration, not a corollary.

Figure 2. Why I1 is non-vacuous: two writers

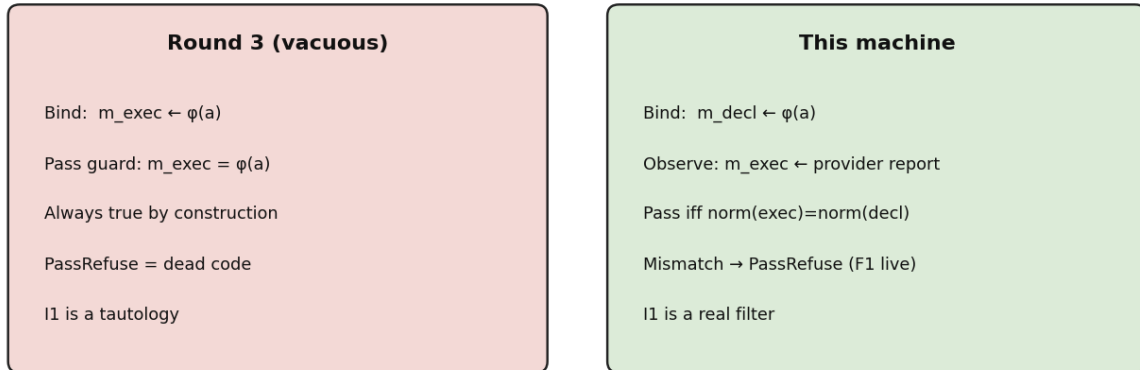
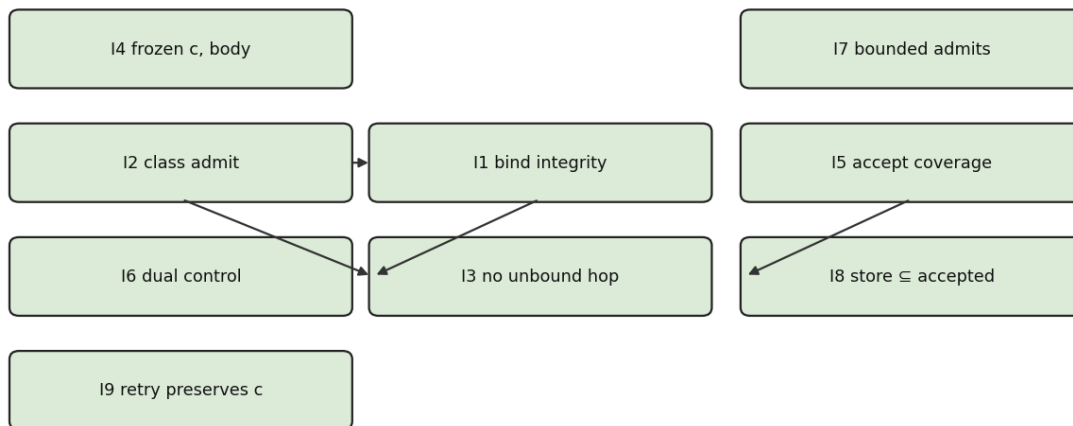


Figure 3. Stage theorem map (I1-I9)



Not stage theorems: quality, item liveness, provider physics, physical context isolation.

| Id | Claim                                                                                                  | Status                 |
|----|--------------------------------------------------------------------------------------------------------|------------------------|
| I1 | $\text{Passed} \Rightarrow \text{norm}(\text{exec}) = \text{norm}(\text{decl}) = \text{norm}(\phi(a))$ | Theorem (Pass-time)    |
| I2 | $\text{Running/Passed} \Rightarrow a \in \pi^* \backslash \delta$                                      | Theorem                |
| I3 | No Pass with exec outside $\phi(\pi^* \backslash \delta)$                                              | Theorem (via I1, I2)   |
| I4 | c and body frozen after Open                                                                           | Theorem                |
| I5 | accepted $\Rightarrow$ every required stage Passed                                                     | Theorem                |
| I6 | check Admit $\Rightarrow a \notin \text{authors}$                                                      | Theorem                |
| I7 | $\leq  \pi^* \backslash \delta $ Admits per stage                                                      | Bound, not termination |

| Id | Claim                                  | Status  |
|----|----------------------------------------|---------|
| I8 | $id \in S \Rightarrow \text{accepted}$ | Theorem |
| I9 | Retry does not write c                 | Theorem |

#### 4.1 Inductive proofs (condensed)

**I1.** Only Pass sets  $pc=Passed$ , and only when  $m\_exec \neq \text{none}$  and  $\text{norm}(m\_exec)=\text{norm}(m\_decl)$ . Bind is the only writer of  $m\_decl$  and writes  $\phi(a)$ . Observe may set a mismatch; then PassRefuse fires and Passed is unreachable. If Bind wrote  $m\_exec$ , the guard would be tautological.

**I2.**  $a$  is written only by Admit, whose guard is  $a \in \pi^* \backslash \delta \backslash \text{tried}$ . Stages of one item are sequential (A4), so  $\pi^*$  is the Admit-time snapshot.

**I3.** I1 gives  $\text{exec} = \phi(a)$ . I2 gives  $a \in \pi^* \backslash \delta$ .

**I4.** No transition after Open writes  $c$  or body.

**I5.** Accept is the only writer of  $\text{status}=\text{accepted}$  and requires all required Passed.

**I6.** Check admit uses  $\pi\_chk = \pi(c) \backslash \text{authors}$  (A6) on the Admit-time snapshot.

**I7.** Each Admit adds a unused specialist to tried (A7). The allow set is finite (A0). Ask may idle; this is a bound on Admit count, not liveness.

**I8.** Accept is the only writer of store  $S$  and sets  $\text{accepted}$  in the same step (A8).

**I9.** Retry does not write  $c$  (I4).

## 5. Context, memory and cache envelope

A work item pins accepted project P and memory K at Open. Each gate Admit creates a monotonic attempt counter, unpredictable nonce and immutable base envelope containing agent/profile, exact provider/API model, instruction hash, context policy, tool authority, FCD cache identity and pre-Admit steering hash. Live steering advances a separate ordered continuation chain.

FCD builds canonical package bytes from include minus exclude, and the adapter independently hashes the bytes it submits; Pass requires current attempt/nonce, package, executor/run, latest steering and executed-model receipts. fresh\_blind excludes author context and forbids executor continuity, and existing executor tools, sessions and provider caches remain external.

| Id  | Context-envelope claim                             |
|-----|----------------------------------------------------|
| I10 | Work P/K and base envelope are pinned              |
| I11 | Pass requires current package receipt              |
| I12 | fresh_blind manifest excludes author context       |
| I13 | Only serialized accepted CAS promotes memory       |
| I14 | No silent project/memory drift                     |
| I15 | Steering stays in scope and latest ack is required |
| I16 | FCD cache is full-identity attempt-local           |
| I17 | Prior-attempt receipts cannot authorize Pass       |

Theorems cover FCD manifests, receipts and state transitions. Adapter honesty, physical prompt isolation, hidden session residue, provider cache neutrality, model quality and impact-review correctness remain explicit assumptions or limits.

## 6. Faults

Each fault is a forbidden transition, not a metaphor.

| Id  | Forbidden step                                                          |
|-----|-------------------------------------------------------------------------|
| F1  | Pass with $\text{norm}(m\_exec) \neq \text{norm}(\phi(a))$              |
| F2  | Two specialists, one runtime instance (needs instance field)            |
| F3  | After $u=0$ , another call without fail-closed, or outside unused allow |
| F4  | Running exit with no published close                                    |
| F5  | $\phi(a)$ not an API identity                                           |
| F6  | Pass with $a \in \delta(c)$                                             |
| F7  | Check admit with $a \in \text{authors}$                                 |
| F8  | Run without a well-formed stage                                         |
| F9  | Stop in chat with status not accepted                                   |
| F10 | Same id, class changes; or retry of $a \in \text{tried}$                |

Weight-sharing across specialists is extra config. It is not in I6.

## 7. Measurement

A named cut is  $[t_0, t_1]$ , with  $W$  the class p95 of completed stage durations in the previous cut, or 12 minutes if  $n < 30$ ; exclude  $t_s > t_1 - W$ , and evaluate  $\pi, \delta, \phi$  as-of event  $t_s$ .

Figure 4. Measurement sample spaces (empty of numbers)

| Rate          | Sample space                     | Event                                                    | Pairing                 |
|---------------|----------------------------------|----------------------------------------------------------|-------------------------|
| Misbind       | first Observe / stage            | $\text{norm}(\text{exec}) \neq \text{norm}(\text{decl})$ | with silent-fail        |
| Silent fail   | well-formed stages + orphan open | no decide/accept in $W$                                  | fail-closed = published |
| Bleed         | stages of class $c$              | $a \notin \pi^*(c)$ as-of $t_s$                          | pair with silent-fail   |
| Time-to-stage | well-formed stages               | survival to decide/accept                                | not a mean              |

No empirical bars. Zeros are proofs only under write-ahead + total call/decide.

Zeros on misbind, bleed and silent-fail are a proof that those faults did not fire only if stage is write-ahead and call/decide are total; otherwise they are estimates biased clean. No numbers in this paper.

## 8. Reference implementation

The control plane is this machine, not a chat host or sessions sidebar. A verified project comes before intake. The operator may choose the gate agent, execution adapter, exact model and context policy before Admit. Agent instructions and exact-route readiness are visible; an unavailable route cannot start. Admit locks the envelope. Existing executors keep their mature tools and session stores; adapters return evidence and receipts but cannot Pass, Accept or promote memory.

The three-pane surface keeps project/capability atlas, selected work line plus bounded gate tray, and real artifact visible together. Questions stay anchored. Steering has explicit project, work, gate, stage, artifact and evidence scope. A pixel is trustworthy only when it projects an event or receipt.

## 9. Related work

Clark and Wilson (1987) already have constrained data items, transformation procedures, a certification relation, integrity verification and mandatory separation of duty: a work item is a CDI, a bound specialist is a TP, and Accept is a validated write.

Saltzer and Schroeder (1975) name fail-safe defaults; deny-override is standard MAC/RBAC; Thomas and Sandhu (1997) bind permission to a task instance. F1/F2/F5 are TOCTOU / control-plane versus data-plane divergence, and F7 is also LLM-as-judge self-preference.

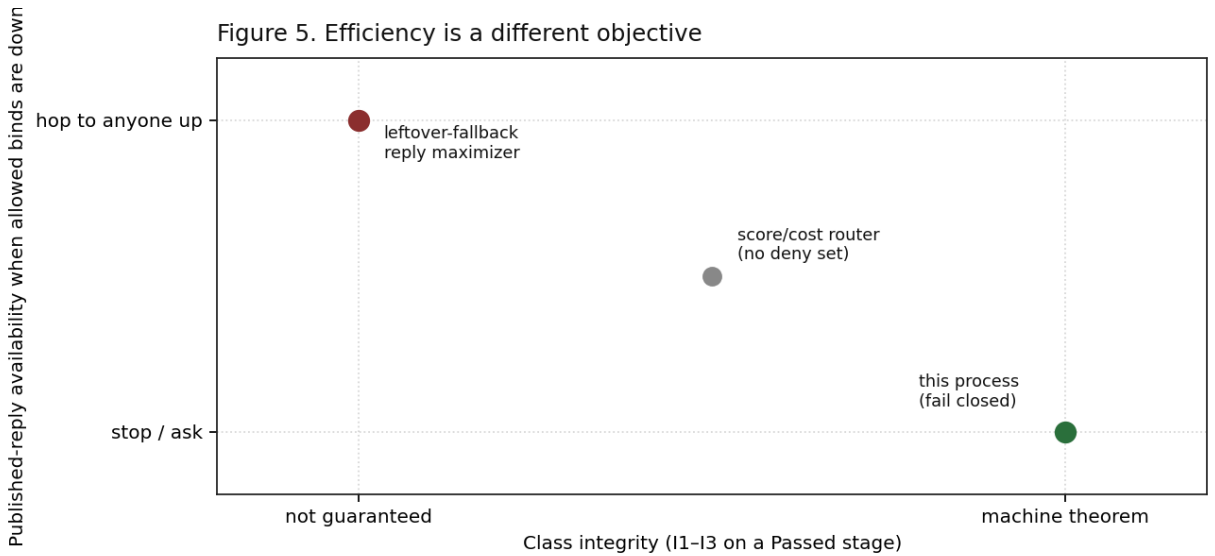
MoE, MoA, FrugalGPT, RouteLLM and MoMA optimize a different objective, and orchestrator-worker (Anthropic, 2024) is a topology. Topology is not the process.

10. Limits

There is no dataset here, no quality theorem, and no item liveness. I1 is the Pass-time report, not execution history, and I1/I3 hold up to norm collision unless norm is injective; I10–I17 prove FCD-owned manifest, receipt, cache and promotion transitions. They do not prove adapter honesty, physical prompt isolation, hidden executor residue, provider cache neutrality or impact-review correctness.

10.1 Efficiency without sacrificing the theorems

Efficiency (tokens, latency, cache hits, published-reply rate) is a different objective from class integrity. A leftover-fallback reply-maximizer improves availability exactly when every allowed bind is down — the same state where this process is already not live. That is not a free lunch. It is a hop.

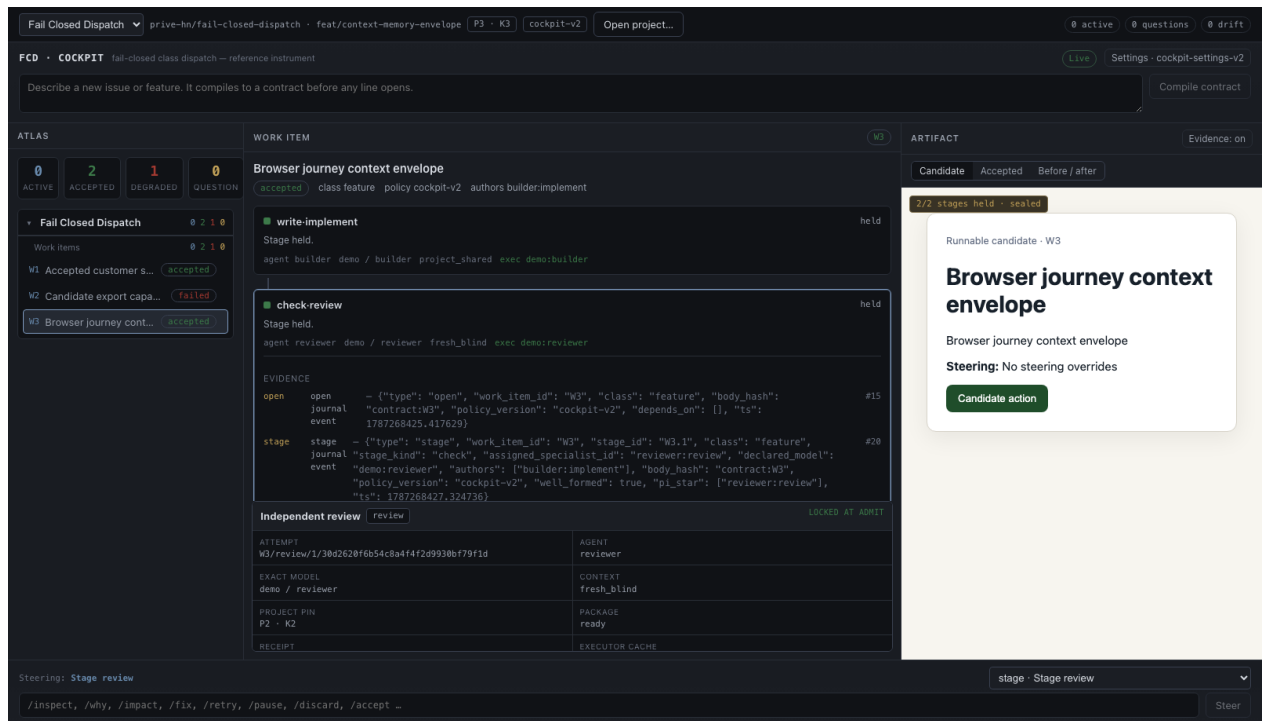


Safe efficiency: (1) FCD cache stays attempt-local and clears on Admit/Close; (2) mature executors may report provider prefix/session reuse, but it has no Pass authority; (3) parallelize only across distinct work items; (4) retry only in  $\pi^*\delta$ tried, bounded by I7; (5) use cheaper models only when they are explicitly selected in policy; (6) Ask instead of hopping when the remainder is empty. These choices preserve I1–I17 under the stated assumptions.

Unsafe efficiency: shared FCD runtime across specialists (F2), session restore that ignores the pinned envelope, cross-attempt FCD cache reuse, leftover fallback, or treating executor-reported cache telemetry as semantic context proof.

11. Interactive evidence layer

The reference implementation keeps verified project/capability atlas, every sibling work line, a bounded editable/locked gate tray and real artifact visible together. Questions stay anchored. Drift review and explicit multi-scope steering remain inside the same cockpit.



The cockpit does not widen the theorem: execution adapters preserve mature worker tools/sessions but cannot Pass, Accept or promote memory; skins are read-only projections; fcd remains the only writer of accepted state. The shown Chrome journey produced package/model receipts before acceptance.

## References

- Anthropic. How we built our multi-agent research system. Anthropic Engineering, 2024.
- Chen, L., Zaharia, M., and Zou, J. FrugalGPT. arXiv:2305.05176, 2023.
- Clark, D. D., and Wilson, D. R. A comparison of commercial and military computer security policies. IEEE Symposium on Security and Privacy, 1987.
- Guo, X., et al. Towards Generalized Routing: MoMA. arXiv:2509.07571, 2025.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. Neural Computation, 1991.
- Ong, I. et al. RouteLLM. 2024.
- Saltzer, J. H., and Schroeder, M. D. The protection of information in computer systems. Proceedings of the IEEE, 1975.
- Shazeer, N. et al. The sparsely-gated mixture-of-experts layer. ICLR, 2017.
- Thomas, R. K., and Sandhu, R. S. Task-based authorization controls (TBAC). 1997.
- Wang, J. et al. Mixture-of-Agents Enhances Large Language Model Capabilities. arXiv:2406.04692, 2024.